Inteligență Artificială

Agenți Inteligenți

Slides: Alexandru Sorici, adaptat după conținut de la Stanford CS221, 2018

- MDP
- Iterarea Valorii/Politicii
- RL

Mediul ca un Proces de Decizie Markov

- **Stări** S, $s_{start} \in S$
- **Acțiuni** A: setul de acțiuni posibile în problema modelată
- $\mathbf{T}(\mathbf{s}, \mathbf{a}, \mathbf{s}')$: funcția de tranziție – modelează probabilitatea $p(s'|s, a)$ de a ajunge în starea s', dacă se execută acțiunea a în starea s
- $\mathbf{R}(\mathbf{s}, \mathbf{a}, \mathbf{s}')$: **recompensa** aferentă tranziției (s, a, s') – valoare care ajută la calcularea **utilității** unui **episod de interacțiune** cu mediul
- $Final(s)$: funcție booleană care stabilește dacă o stare s este una care **termină** un episod de interacțiune în mediu
- **Factor de atenuare (discount)**: $0 \leq \gamma \leq 1$ – factor ce permite *atenuarea* influenței recompensei din viitor în favoarea celei din prezent

Caracteristici MDP:

- Caracteristică funcție de tranziție:
  - **Proprietatea Markov**: $s_t \perp s_{t-2} | s_{t-1} \Rightarrow T(s_{t-1}, a, s_t) = p(s_t | s_{t-1}, a_{t-1})$ – o stare încapsulează toată informația asupra situația instantanee a mediului, astfel că tranziția de la o stare curentă la una succesoare depinde doar de starea curentă și acțiunea luată
  - $\forall s, \forall a : \sum_{s' \in S} T(s, a, s') = 1$
- Existența unui **semnal de recompensă** care este utilizat pentru a putea exprima **funcții de utilitate** ale agenților din mediu.
  - Desemnarea valorilor de recompensă pentru fiecare stare face parte din **proiectarea** mediului MDP

Obiectivul unui agent într-un MDP

- **Găsirea politicii optime** π^*, unde $\pi : S \rightarrow A$ – funcție care asignează fiecărei stări o acțiune de executat ("plan" al agentului)

Pentru a găsi politica optimă, trebuie definită o metodă de **evaluare** a unei politici date

Definiții:

- Utilitatea unei politici π este suma (atenuată – discounted) a **recompenselor** primite pe secvența de interacțiuni cu mediul (calea acesta este o **variabilă aleatoare**)

Calea unui episod: $s_0, a_1, \mathbf{r_1}, s_1, a_2, \mathbf{r_2}, \ldots$ (acțiune, recompensă, stare nouă)

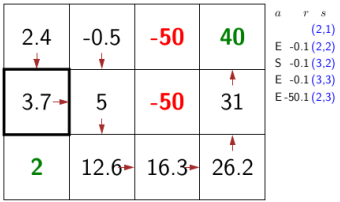
Utilitatea (eng. Gain) unui episod, sub factorul de atenuare γ este:

$$G_1 = u_1 = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \ldots$$

Cale	γ	u
[in joc; cont, 4, final]	1	4
[in joc; cont, 4, in joc; cont, 4, in joc, cont, 4, final]	1	12
[in joc; cont, 4, in joc; cont, 4, final]	1	8
[in joc; cont, 4, final]	0	4
[in joc; cont, 4, in joc; cont, 4, in joc, cont, 4, final]	0	4
[in joc; cont, 4, in joc; cont, 4, final]	0	4
[in joc; cont, 4, final]	1/2	4
[in joc; cont, 4, in joc; cont, 4, in joc, cont, 4, final]	1/2	7
[in joc; cont, 4, in joc; cont, 4, final]	1/2	6

Definiții:

- Valoarea unei politici π **într-o stare** S este **utilitatea acelei stări** în valoare așteptată.



$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots | s_t = s\right] = \mathbb{E}_{\pi}\left[\sum_{\tau=t+1} \gamma^{\tau-t-1} r_{\tau} | s_t = s\right] = \mathbb{E}_{\pi}\left[G_t | s_t = s\right]$$

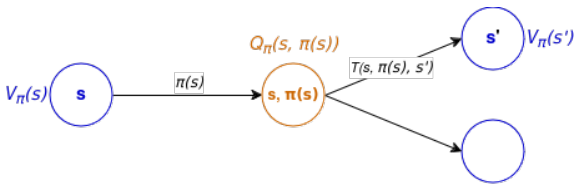
Ecuații Bellman

- Definesc o recurență (în stil programare dinamică) între $V_\pi(s_t)$ și $V_\pi(s_{t+1})$ pe principiul împărțirii calculului utilității politicii în: (1) **recompensa imediata** plus (2) **suma atenuata a recompenselor viitoare (estimate)** din starea urmatoare

- *Principle of Optimality: An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision. (Bellman, 1957)*

Definiții:

- $V_\pi(s)$ – utilitatea **în valoare așteptată** cumulată ca urmare a utilizării politicii π din starea s
- $Q_\pi(s, a)$ – utilitatea **în valoare așteptată** cumulată ca rezultat al alegerii acțiunii a în starea s, folosind politica π



$$V_\pi(s) = \begin{cases} 0 & daca\ Final(s) \\ Q_\pi(s, \pi(s)) & altfel \end{cases} \qquad Q_\pi(s, a) = \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma V_\pi(s') \right]$$

Idee: definire algoritm iterativ:

- Inițializează valori arbitrare pentru politică
- Aplică recurența dată de ecuațiile Bellman până la convergență

Algoritm: Evaluarea Politicii

Intrări: $< S, A, T, R, \gamma >$ – parametrizare MDP;
π – politică de parcurgere a MDP-ului;

Ieșire: V_π: Valoarea estimată a fiecărei stări $s \in S$ sub politica π

1. Inițializare $V_\pi^{(0)}(s) \leftarrow 0$ **pentru** toate stările $s \in S$
2. **pentru fiecare** iterație $t = 1, \ldots, N$:
3. **pentru fiecare** stare s:
4. $$V_\pi^{(t)}(s) \leftarrow \underbrace{\sum_{s'} T(s, \pi(s), s') \left[R(s, \pi(s), s') + \gamma V_\pi^{(t-1)}(s') \right]}_{Q_\pi^{(t-1)}(s, \pi(s))}$$
5. **întoarce** V_π

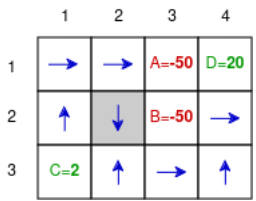
Caracteristici implementare algoritm de evaluare a politicii:

- Iterațiile pot continua până la o limită preimpusă (N) sau până când valorile V_π nu se mai schimbă mult:

$$\max_{s \in S} |V_\pi^{(t)} - V_\pi^{(t-1)}| \leq \epsilon$$

- Nu e nevoie să stocăm $V_\pi^{(t)}$ pentru fiecare iterație t, avem nevoie doar de ultimele două: $V_\pi^{(t)}$ și $V_\pi^{(t-1)}$

Fie exemplul următor, unde recompensa default (pentru mișcare este 0), iar săgețile albastre indică politica de urmat π_0:



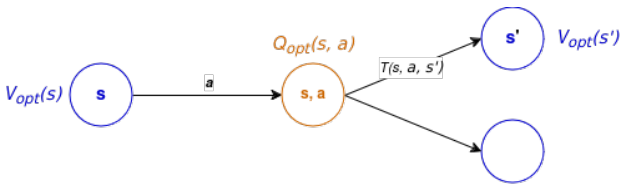
	1	2	3	4
1	-39.27	-45.55	A=-50	D=20
2	-32.66	-16.11	B=-50	9.60
3	C=2	-11.35	0.56	7.65

Valorile $V_{\pi_0}^{(0)}$ sunt inițial 0 pentru toate stările; $\gamma = 0.9$
Calculăm actualizarea pentru starea $(2,2)$:

$$V_{\pi_0}^{(1)}((2,2)) = \sum_{s'} T((2,2),\downarrow,s') \left[R((2,2),\downarrow,s') + \gamma V_{\pi}^{(0)}(s')\right]$$

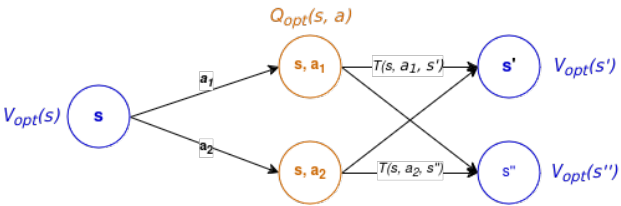
$$= 0.8 \cdot (0 + 0.9 \cdot 0) \quad \triangleright \text{ merg jos}$$
$$+ 0.1 \cdot (0 + 0.9 \cdot 0) \quad \triangleright \text{ alunec st.}$$
$$+ 0.1 \cdot (-50 + 0.9 \cdot 0) \quad \triangleright \text{ alunec dr.}$$
$$= -5.0$$

Găsirea politicii optime și a valorii optime



- Algoritmul de evaluare a valorii unei politici menține constantă acea politică pe parcursul iterației, dar asta **nu** ne indică dacă acea politică este **cea mai bună** de urmat
- Dorim să găsim politica optimă (π^*), care maximizează valoarea așteptată a fi acumulată (V^* sau V_{opt}) din fiecare stare $s \in S$

- Notăm prin $V_\pi(s)$ și $Q_\pi(s,a)$ funcția de valoare și funcția Q **aferente politicii** π
- Notăm prin $V^*(s) = V_{opt}(s)$ și $Q^*(s,a) = Q_{opt}(s,a)$ funcțiile de valoare și Q **aferente politicii optime** π^*



$$V_{opt}(s) = \begin{cases} 0 & daca\ Final(s) \\ \max_{a \in A} Q_{opt}(s,a) & altfel \end{cases}$$

$$Q_{opt}(s,a) = \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_{opt}(s') \right]$$

$$\pi^* = arg \max_{a \in A} Q_{opt}(s,a)$$

Iterarea Politicii

- Algoritmul **Iterarea Politicii** găsește o politică optimă alternând pași de **evaluare** și de **îmbunătățire** a politicii.

$$\pi_0 \longrightarrow V_{\pi_0} \longrightarrow \pi_1 \longrightarrow V_{\pi_1} \longrightarrow \pi_2 \longrightarrow \ldots \longrightarrow \pi^*$$

Algoritm: Iterarea Politicii

Intrări: $\langle S, A, T, R, \gamma \rangle$ – MDP

Ieșire: V_{opt}, π^*

1. Inițializare: $\pi(s) \leftarrow rand(A)$, $V(s) \leftarrow 0$ pentru fiecare $s \in S$; $terminat \leftarrow fals$
2. **cât timp** $\neg terminat$
3. $\quad V \leftarrow EvaluarePolitică(\langle S, A, T, R, \gamma \rangle, \pi)$
4. $\quad$ **pentru fiecare** $s \in S$:
5. $\quad\quad a_{old} \leftarrow \pi(s)$
6. $\quad\quad \pi(s) \leftarrow arg\max_{a \in A} \sum_{s' \in S} T(s, a, s') \left[R(s, a, s') + \gamma V(s') \right]$
7. $\quad\quad terminat \leftarrow terminat \wedge a_{old} == \pi(s)$
8. **întoarce** V, π

Iterarea Valorii

- **Iterarea Valorilor** combină **într-un singur pas** (fără a pierde garanția convergenței) operațiile de **evaluare** și **îmbunătățire** a politicii

Algoritm: Iterarea Valorilor

Intrări: $\langle S, A, T, R, \gamma \rangle$ – MDP; ϵ – prag de convergență

Ieșire: V_{opt}, π^*

1. Inițializare $V^{(0)}(s) \leftarrow 0$ pentru toate s;
2. **repetă**
3. $\quad \delta \leftarrow 0$
4. $\quad$ **pentru fiecare** $s \in S$:
5. $\quad\quad V^{(t)}(s) \leftarrow \max_{a \in A} \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V^{(t-1)}(s') \right]$
6. $\quad\quad \delta \leftarrow max(\delta, |V^{(t)}(s) - V^{(t-1)}(s)|)$
7. **până când** $\delta < \epsilon$
8. **pentru fiecare** $s \in S$:
9. $\quad \pi(s) \leftarrow arg \max_{a \in A} \sum_{s' \in S} T(s,a,s') \left[R(s,a,s') + \gamma V(s') \right]$ ▷ extragere politică optimă
10. **întoarce** $V^{(t)}$, π

- **Iterarea Valorilor** este garantat să conveargă atâta timp cât: $\gamma < 1$ sau **graful MDP** este **aciclic**

Exemplu: un pas de IV pe gridul cu vulcanul

	1	2	3	4
1			A=-50	D=20
2			B=-50	
3	C=2			

	1	2	3	4
1	0.0	0.0	A=-50	D=20
2	1.60	0.0	B=-50	11.00
3	C=2	1.60	0.0	0.0

Valorile $V_{opt}^{(0)}$ sunt inițial 0 pentru toate stările; $\gamma = 0.9$
Calculăm actualizarea pentru starea $(2, 4)$:

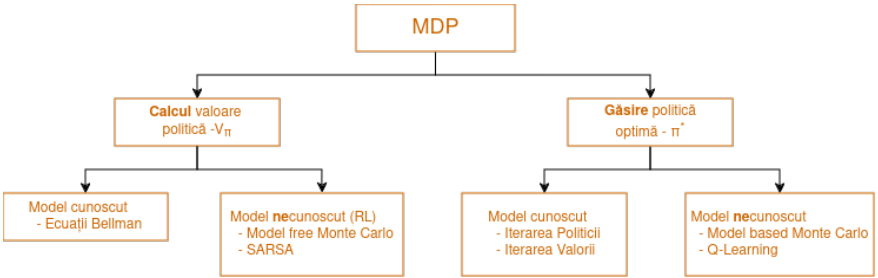
$$V_{opt}^{(1)}((2,4)) = \max_{a \in A} \sum_{s'} T((2,4), a, s') \left[R((2,4), a, s') + \gamma V_{opt}^{(0)}(s') \right]$$

$$\begin{aligned} a_{max} = \ & \uparrow \\ & = 0.8 \cdot (20 + 0.9 \cdot 0) \quad \triangleright \text{ merg sus} \\ & + 0.1 \cdot (-50 + 0.9 \cdot 0) \quad \triangleright \text{ alunec st.} \\ & + 0.1 \cdot (0 + 0.9 \cdot 0) \quad \triangleright \text{ alunec dr.} \\ & = 16 - 5 = 11 \end{aligned}$$

- Algoritmii **Iterarea Politicii** / **Valorii** presupun **cunoașterea dinamicii mediului** ($T(s, a, s') = p(s'|s, a)$)

- În multe situații reale, dinamica mediului **nu poate fi** cunoscută în avans
  - Agentul trebuie **să adune cunoștințe** explorând mediul ⇒ Învățarea prin Recompensă (Reinforcement Learning – RL)

- **MDP cu dinamică știută**: **extragerea** politicii optime se face printr-o procedură ***offline*** (fără a face *efectiv* acțiuni în mediu)
- **RL pe un MDP**: politica optimă **"se învață"** ***online*** prin **executarea** efectivă a unor **acțiuni în mediu** și **colectarea de recompense**



Un algoritm de RL are nevoie de:

- O politică **după care învață** să estimeze **valorile stărilor**
- O politică **după care explorează** mediul

Cele doua politici pot fi identice (**on-policy learning**) sau diferite (**off-policy learning**)

Algorithms

Evaluating a given policy π

Known model

Bellman's eqns

$$V_\pi^{(t)}(s) \leftarrow \underbrace{\sum_{s'} T(s, \pi(s), s')[\text{Reward}(s, \pi(s), s') + \gamma V_\pi^{(t-1)}(s')]}_{Q^{(t-1)}(s, \pi(s))}$$

Unknown model

Algorithm: SARSA

On each (s, a, r, s', a'):

$$\hat{Q}_\pi(s, a) \leftarrow (1-\eta)\hat{Q}_\pi(s, a) + \eta[\underbrace{r}_{\text{data}} + \gamma \underbrace{\hat{Q}_\pi(s', a')}_{\text{estimate}}]$$

Algorithm: model-free Monte Carlo

On each (s, a, u):

$$\hat{Q}_\pi(s, a) \leftarrow (1-\eta)\hat{Q}_\pi(s, a) + \eta \underbrace{u}_{\text{data}}$$

Computing optimal policy π^{opt}

Known model

Value iteration

$$V_{\text{opt}}^{(t)}(s) \leftarrow \underbrace{\max_{a \in \text{Actions}(s)} \sum_{s'} T(s, a, s')[\text{Reward}(s, a, s') + \gamma V_{\text{opt}}^{(t-1)}(s')]}_{Q_{\text{opt}}^{(t-1)}(s, a)}$$

Unknown model

Algorithm: Q-learning [Watkins/Dayan, 1992]

On each (s, a, r, s'):

$$\hat{Q}_{\text{opt}}(s, a) \leftarrow (1-\eta)\underbrace{\hat{Q}_{\text{opt}}(s, a)}_{\text{prediction}} + \eta\underbrace{(r + \gamma \hat{V}_{\text{opt}}(s'))}_{\text{target}}$$

Recall: $\hat{V}_{\text{opt}}(s') = \max_{a' \in \text{Actions}(s')} \hat{Q}_{\text{opt}}(s', a')$

Model based MC

$$\hat{T}(s, a, s') = \frac{\#\text{ times } (s, a, s') \text{ occurs}}{\#\text{ times } (s, a) \text{ occurs}}$$

$$\widehat{\text{Reward}}(s, a, s') = r \text{ in } (s, a, r, s')$$

RL : Model-based RL

- Model-based Monte Carlo – idee de bază: dacă nu avem un model vom **estima** unul, în particular $\hat{T}(s, a, s')$ și $\hat{R}(s, a, s')$

- Exemplu: pentru episoade care sunt de forma $s_0; a_1, r_1, s_1; a_2, r_2, s_3; \ldots a_n, r_n, s_n$
  - Tranziții: $\hat{T}(s, a, s') = \frac{aparitii(s,a,s')}{aparitii(s,a)}$, unde $aparitii()$ este o funcție cu rol de contorizare
  - Recompense: $\hat{R}(s, a, s') = aparitii_r(s, a, r, s')$ (de cate ori apare valoarea r în tranziții de la s la s' prin a)

Probleme:

- Dacă secvența: $s_0; a_1, r_1, s_1; a_2, r_2, s_3; \ldots a_n, r_n, s_n$ se obține **urmând o politică dată** π atunci e posibil să nu vedem niciodată selecția (s, a) dacă $\pi(s) \neq a$

$\Rightarrow$ **nevoia** ca π să **exploreze** în mod explicit

RL : Model-free Monte Carlo

- Să presupunem că **nu** dorim să ne modelăm mediul
- Dorim să estimăm $Q_{opt}(s, a)$ direct din interacțiuni cu mediul și obținere de recompense
- **Reminder:** $Q^{\pi}(s, a) = \mathbb{E}_{\pi}\left[\underbrace{r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots}_{u_{t+1}} | s_t = s, a_t = a\right]$
- Atunci, **estimarea** $\hat{Q}_{\pi}(s, a)$ se poate face ca **medie** a lui $\mathbf{u_{t+1}}$ atunci când $s_t = s$ și $a_t = a$
  - Altfel spus, **explorăm** cu politici diferite (care au în componență elemente de alegere aleatoare a unor acțiuni într-o stare) mediul, obținând **episoade** de utilitate (u_0 – utilitatea acumultată începând din starea s_0) diferite
  - Facem media acestor episoade pentru fiecare decizie de aplicare a lui a în s

RL : Model-free Monte Carlo

Formulări echivalente a problemei de estimare a lui $\hat{Q}_\pi(s, a)$

- Formulare inițială: $\hat{Q}_\pi(s, a)$ = **medie** a lui $\mathbf{u_{t+1}}$ atunci când $s_t = s$ și $a_t = a$
- Formulare ca medie mișcătoare (interpolare) (*moving average*). Fie $\eta = \frac{1}{1+\#\text{actualizari}(s,a)}$
  - Pentru fiecare (s, a, **u**)
    - $\hat{Q}_\pi(s, a) \leftarrow (1 - \eta) \cdot \hat{Q}_\pi(s, a) + \eta \cdot u$
- Formulare ca *eroare de predicție*:
  - Pentru fiecare (s, a, **u**)
    - $\hat{Q}_\pi(s, a) \leftarrow \hat{Q}_\pi(s, a) + \eta \cdot (\underbrace{u}_{\textit{target}} - \underbrace{\hat{Q}_\pi(s, a)}_{\textit{predicție}})$

u este, în fiecare caz, utilitatea concret câștigată în cadrul episodului în care se urmează politica π

Învățare prin Diferențe Temporale

- Mecanism de învățare întâlnit și în regnul animal
- Idee de **actualizare** a unei predicții curente prin: (1) **obținerea unei informații noi concrete** și (2) efectuare unei (celei mai bune) **predicții pentru viitor** $\longrightarrow$ procedură de **bootstrapping**

Formulare **Model-free Monte Carlo** ca *eroare de predicție*:

- Pentru fiecare (s, a, **u**)
  - $\hat{Q}_\pi(s,a) \leftarrow \hat{Q}_\pi(s,a) + \eta \cdot (\underbrace{u}_{target} - \underbrace{\hat{Q}_\pi(s,a)}_{predicție})$

Formulare **SARSA (state-action-reward-state-action)**:

- Pentru fiecare (s, a, r, s', a')
  - $\hat{Q}_\pi(s,a) \leftarrow \hat{Q}_\pi(s,a) + \eta \cdot (\underbrace{r + \gamma \hat{Q}_\pi(s',a')}_{target} - \underbrace{\hat{Q}_\pi(s,a)}_{predicție})$

RL : Provocări și Aplicații

RL provocări:

- Compromis între **explorare** și **exploatare**: **cum setez** η
- Cum **proiectez** MDP-ul: cum se face **proiectarea recompenselor**
- Cum invit la **explorare intrinsecă** (explorare de dragul explorării)

RL în lumea reală:

- În probleme care implică predicția **unui control optim** *adaptabil* la un mediu de operare dat
- În probleme în care se dorește includerea **unui feedback** + învățare pe parcurs (online)
- În probleme în care **obținerea "setului de date"** / **cunoștințelor** nu se poate face decât **explorând**

RL : Provocări și Aplicații

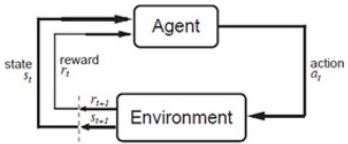
Google DeepMind – dezvoltare a principiilor de RL + DeepLearning pe seama jocurilor Atari



Sursa: Google DeepMind

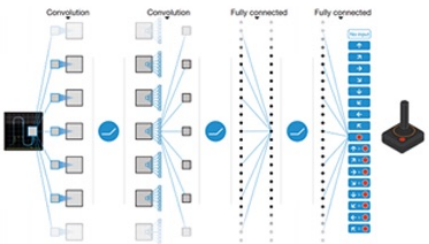
RL : Provocări și Aplicații

Google DeepMind – dezvoltare a principiilor de RL + DeepLearning pe seama jocurilor Atari



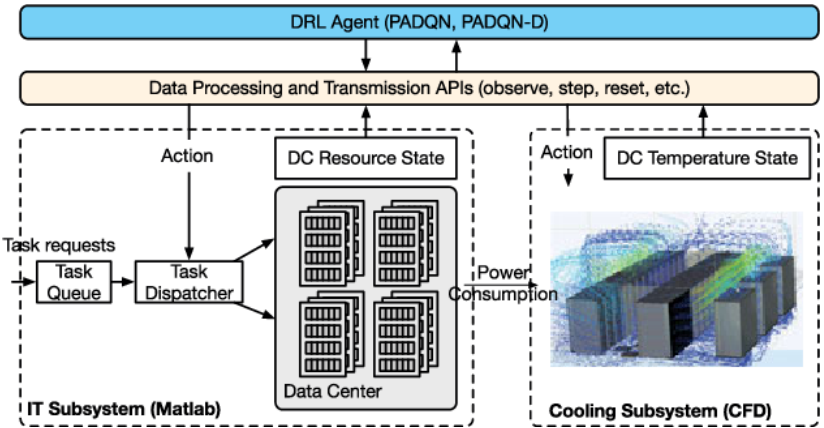






Sursa: opendatascience.com

Operare Eficientă Centru de Date



Sursa: Optimizing Energy Efficiency for Data Center via Parameterized Deep Reinforcement Learning, IEEE Transactions on Services Computing, 2023

RL : Provocări și Aplicații

Google DeepMind: control al câmpurilor magnetice într-un reactor de fuziune nucleară



Control al plasmei în reactoare Tokamak pentru fuziune nucleară:
https://deepmind.google/discover/blog/accelerating-fusion-science-through-learned-plasma-control/

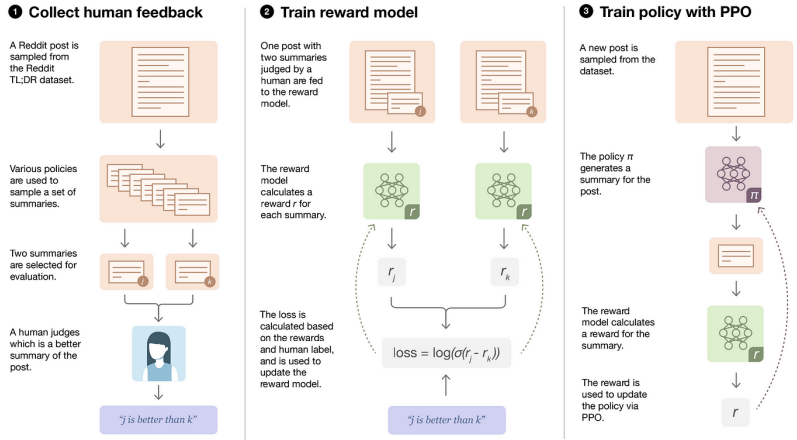
RL : Provocări și Aplicații

Optimizarea plasării de reclame personalizate în sisteme de recomandare



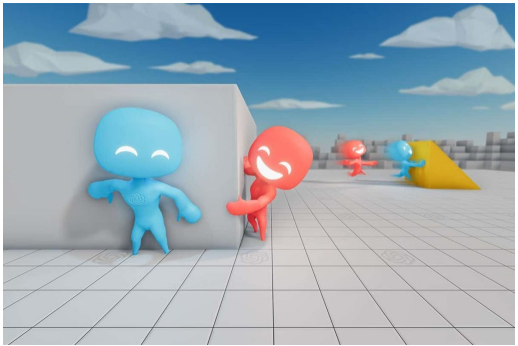
Sursa: Zhao et al., DEAR: Deep Reinforcement Learning for Online Advertising Impression in Recommender Systems, 2021

Reinforcement Learning from Human Feedback – aliniere răspunsuri LLM cu preferințe umane



Sursa: Isaac Kargar, AIGuys, Medium

Agenți și Sisteme Multi-Agent – MARL



OpenAI Hide and Seek

Mulțumesc!



Feedbackul este binevenit!